Team Workstreams

Three people, three owners. Each owner has a clear area, a list of tasks, and a definition of done. Swap names in: **Dev 1** (GitHub integration), **Dev 2** (monitoring and failure intelligence), **Dev 3** (testing and quality).

Goal for the first 8 weeks: a failed GitHub Actions deployment is detected, diagnosed, and (when it is a code or config problem) turned into a validated fix pull request that a human approves. See `docs/product/PLAN.md` for the evaluation set and targets.

---

1. Who owns what

	Dev 1: GitHub integration	**Dev 2: Monitoring and failure intelligence**	**Dev 3: Testing and quality**
Mission	Connect GitHub, receive failures, create PRs	Understand failures: logs, redaction, classification, dashboards	Prove it works: test repo, failure set, eval harness, CI, sandbox
Main code	`apps/api` (auth, github, projects, webhooks)	`apps/worker` (failure pipeline), `apps/web` (failures views)	`failure-testbed` repo, `apps/eval`, CI workflows, test utilities
Hands off to	Dev 2 (failure events, GitHub client)	Dev 1 (fix proposals to turn into PRs)	Everyone (failure repos, results, CI)

Two adjustments to your original split, and why:

1. **"Monitoring" starts as pipeline and deployment monitoring.** For the MVP, monitoring means seeing failed runs, their logs, and their diagnosis in the dashboard. Prometheus, Grafana, custom graphs and alerting come after the week 8 decision.
2. **Dev 2 also owns the AI classifier.** Failure detection feeds the AI directly, so the same person should own both. This is the riskiest part, so Dev 2 starts on it in week 1.

---

2. Contracts between you (agree in week 1, half a day together)

These are the handoff points. Put each as a Zod schema in `packages/shared` and do not change them without telling the others.

Contract	From → To	Content
`FailureDetected` job	Dev 1 → Dev 2	`failureId`, `projectId`, `repoFullName`, `runId`, `headSha`, `branch`, `workflowName`
`GitHubClient` service	Dev 1 → Dev 2	`getFailedJobLogs(failureId)`, `getFileContents(repo, path, ref)`, `listRecentCommits(repo, ref)`
`FixProposal` object	Dev 2 → Dev 1	`failureId`, `classification`, `outcome` (`fix` / `diagnosis` / `retry`), `patch` (unified diff, optional), `explanation`, `confidence`
Testbed `meta.json`	Dev 3 → Dev 2	`id`, `slug`, `class`, `expectedOutcome`, `description`, `expectedFiles`
Eval result row	Dev 3 → all	`failureId`, `classifiedCorrectly`, `outcomeCorrect`, `validationPassed`, `wouldMerge`, `secretLeak`, `minutes`

3. Weekly milestones

Week	Dev 1	Dev 2	Dev 3	Integration checkpoint
1	GitHub App, login with GitHub	Log cleanup + redaction on sample logs, failures page shell	Create testbed repo, baseline app, first 5 failures, CI for main repo	Contracts agreed. Testbed workflow runs green on `main` and red on failure branches
2	Connect repo, projects, webhook endpoint, `failures` table	Classifier v1 on collected logs	Remaining failures (20 total), webhook test fixtures	Break a testbed branch → failure row appears in the app
3	`GitHubClient`: logs, files, commits	Classifier accuracy report, failures list and detail pages	Eval harness v1 (classification only)	First eval run: classification accuracy number
4-5	PR creation service	Fix agent + context builder	Sandbox validator (apply patch, rerun build and tests in a container)	A fix PR is opened for an easy failure
6-7	Approval actions (approve, reject, request change), audit log	Failure detail with proposed patch and validation result	Eval harness v2 (full scoring), more test coverage	Full flow works for 3 failures
8	Demo prep	Prompt and context fixes for worst cases	Full run of all 20, final scores	Go / change approach / narrow, decided together

Targets at week 8: classification ⩾ 80%, validated fix PRs ⩾ 30% of rows 1-17, zero secret leaks.

---

4. Working agreements

- Follow `AGENTS.md` and the skills in `.claude/skills/`. One task, one task file in `tasks/`, one branch, one PR.
- Run `pnpm tasks:list` at the start of each day. Check overlaps before starting.
- Open a PR for every change. Another teammate reviews. Rebase on `main` before opening.
- When a contract needs to change, post it in the team chat first, then update the shared schema in its own small PR.

- Short sync (15 minutes, same time daily or every other day): what is done, what is next, what is blocked.

---

5. Dev 1: GitHub integration

Goal: a user logs in with GitHub, connects a repo, and the platform receives and stores failed workflow runs, can read logs and files, and can open pull requests.

Steps and to-dos

Step 1. Create the GitHub App (do this first, everyone depends on it)

- [ ] GitHub → Settings → Developer settings → GitHub Apps → New GitHub App (under your organization, not a personal account).
- [ ] Webhook URL: your tunnel URL + `/api/webhooks/github`. Generate a webhook secret.
- [ ] Repository permissions: **Contents** read and write, **Pull requests** read and write, **Actions** read, **Metadata** read.
- [ ] Subscribe to events: **Workflow run**.
- [ ] Enable "Request user authorization (OAuth) during installation" and set the callback URL to the API.
- [ ] Generate a private key (`.pem`). Never commit it. Store the App ID, client ID, client secret, webhook secret, and key in `.env` (add placeholders to `.env.example`).
- [ ] Set up a tunnel (Cloudflare Tunnel or ngrok) with a stable URL, and document it in `docs/guides/github-app-setup.md`.

Step 2. Login with GitHub

- [ ] Add auth in `apps/api` (`modules/auth`): GitHub OAuth, session or JWT, `GET /api/auth/me`.
- [ ] Store users in the `User` table (GitHub id, login, avatar). Never store tokens in plain text.
- [ ] Web: login button, callback handling, logged-in header with the user name.
- [ ] Test: login works, logout works, protected endpoint returns 401 when logged out.

Step 3. Installations and repo connect

- [ ] Handle the installation flow and store the installation id per organization.
- [ ] `GitHubModule` with Octokit using GitHub App authentication to get short-lived installation tokens.
- [ ] `GET /api/github/repos`: list repos available to the installation.
- [ ] `POST /api/projects`: save a chosen repo as a project (belongs to an organization).
- [ ] Web: "Connect repo" page and a project list page.

Step 4. Webhook endpoint

- [ ] `POST /api/webhooks/github`: read the raw body, verify `X-Hub-Signature-256` (HMAC SHA-256 with the webhook secret). Reject invalid signatures with 401.

- [ ] Handle only `workflow_run` events with `action = completed` and `conclusion = failure`. Ignore everything else with 200.
- [ ] De-duplicate using the `X-GitHub-Delivery` header and the run id, so a redelivered webhook creates one failure.
- [ ] Save a `Failure` row, then enqueue the `FailureDetected` job (contract above).
- [ ] Tests: valid signature, invalid signature, duplicate delivery, non-failure event.

Step 5. GitHubClient for Dev 2

- [ ] `getFailedJobLogs(failureId)`: list jobs of the run, find the failed ones, download job logs. Return text per job and step name.
- [ ] `getFileContents(repo, path, ref)` and `listRecentCommits(repo, ref)`.
- [ ] Handle rate limits and 404s with clear errors.

Step 6. Pull request creation (weeks 4-5)

- [ ] Input: a `FixProposal` with a patch. Create a branch from the failing commit, apply the patch, commit, open a PR.
- [ ] PR description contains: classification, explanation, log excerpt (redacted), validation result.
- [ ] Never push to the user's default branch.
- [ ] Tests with a fake GitHub client, plus one real run against the testbed repo.

Step 7. Approval actions and audit (weeks 6-7)

- [ ] Approve, reject, and request-change actions on a fix attempt, stored with who and when.
- [ ] Audit log entry for every AI action and every human decision.

Definition of done

Break a testbed branch → the failure appears in the app within seconds, its logs can be fetched, and (later) a PR appears on the testbed repo.

---

6. Dev 2: Monitoring and failure intelligence

Goal: turn a raw failure into a safe, useful diagnosis, show it in the dashboard, and later produce fix proposals.

Steps and to-dos

Step 1. Collect real logs first (week 1, before the platform exists)

- [ ] Get the first failing testbed branches from Dev 3, download their logs from GitHub Actions, and save them in `apps/worker/test/fixtures/logs/` (fake data only).
- [ ] Use these as input for every step below.

Step 2. Log cleanup

- [ ] `log-cleaner` in `apps/worker/src/failures/`: strip ANSI codes and timestamps, keep the failing step, extract the error lines plus some context lines, enforce a maximum size.
- [ ] Tests with the fixture logs.

Step 3. Redaction (must exist before any text reaches a model)

- [ ] `redactor`: remove GitHub tokens (`ghp_`, `ghs_`), AWS keys, private key blocks, JWTs, bearer tokens, `KEY=value` style secrets, emails, and IP addresses.
- [ ] Tests: each pattern, plus a test that fails if any known fake secret in the fixtures survives.
- [ ] Rule: the AI client only accepts redacted text (enforce with a type, e.g. `RedactedText`).

Step 4. Classifier

- [ ] Worker processor for `FailureDetected`: fetch logs via `GitHubClient`, clean, redact, classify.
- [ ] Classes: `code`, `dependency`, `config`, `infra`, `secrets`, `flaky`. Use structured output validated by Zod, with a reason and a confidence value.
- [ ] Store the diagnosis on the failure.
- [ ] Run it against all collected logs and record accuracy against Dev 3's `meta.json`. Target ≥ 80% before building the fix agent.

Step 5. Failures dashboard (monitoring v1)

- [ ] Web `features/failures/`: list with status and class, filters, detail page showing redacted logs, classification, reason, and later the proposed patch and validation result.
- [ ] Project overview with counts: failures by class, fix PRs opened, merged, rejected.

Step 6. Fix agent (weeks 4-5, only after Step 4 passes)

- [ ] Context builder: failing files named in the log, recent commits, config files (Dockerfile, workflow, manifests). No secret values.
- [ ] Fix agent returns a `FixProposal`. For `infra`, `secrets`, and `flaky` it must return `outcome = diagnosis` or `retry` with guidance and no patch.
- [ ] Pass the patch to Dev 3's validator, and attach the result.

Step 7. After the week 8 decision (not before)

- [ ] Prometheus and Grafana provisioning, the Metrics Gateway, custom widgets, alerting tabs (see `docs/architecture/`).

Definition of done

A failed run produces a stored, redacted, correctly classified diagnosis visible in the dashboard, with no secrets in any model input.

---

7. Dev 3: Testing and quality

Goal: a trustworthy way to measure the product. A separate test repo with intentional failures, an eval harness that scores results, CI for the main repo, and a sandbox that validates fixes.

Part A. Create the new testing repo (`failure-testbed`)

Why a separate repo: the GitHub App needs real repos with real Actions runs, and the intentional failures must not pollute the product repo.

A1. Create and protect the repo

- ☐ In your GitHub organization: New repository → name `failure-testbed`.
- ☐ Visibility: public is fine, since it holds only fake sample code and no secrets. It also avoids using up private Actions minutes. Use private if you prefer.
- ☐ Add a README, a `.gitignore` for Node, and an MIT license (optional).
- ☐ Settings → Branches: protect `main` (require PR). Failure branches `fail/**` are not protected.
- ☐ Add teammates as collaborators or organization members.
- ☐ Install the GitHub App (made by Dev 1) on this repo.

A2. Clone and create the structure

bash

```
git clone git@github.com:<org>/failure-testbed.git
cd failure-testbed
mkdir -p app/src app/test app/k8s failures scripts .github/workflows
```

```
failure-testbed/
├─ app/                          # small healthy Node + TypeScript API
│  ├─ src/                       # index.ts, routes/health.ts, math.ts
│  ├─ test/                      # unit tests
│  ├─ k8s/                       # deployment.yaml, service.yaml
│  ├─ Dockerfile
│  ├─ package.json, tsconfig.json
├─ .github/workflows/ci-deploy.yml
├─ failures/
│  ├─ 01-typo-import/            # patch.diff + meta.json
│  └─ ...
├─ scripts/
│  ├─ create-branches.sh         # makes fail/NN-slug branches from main
│  ├─ verify-patches.sh          # checks every patch applies cleanly to main
│  └─ reset-branches.sh          # deletes and recreates failure branches
└─ README.md
```

A3. Build the healthy baseline app

- [ ] Node 22 + TypeScript + Express with `GET /health`, a few functions (for example `math.ts`), and 5-10 unit tests that pass. Use the same test runner your main repo uses.
- [ ] Scripts in `package.json`: `lint`, `typecheck`, `test`, `build`, `start`.
- [ ] `Dockerfile` that builds and starts the app on a fixed port.
- [ ] Kubernetes `deployment.yaml` and `service.yaml` with a readiness probe on `/health`.

A4. Build the workflow (`ci-deploy.yml`) Triggers: push to `main` and `fail/**`. Steps:

1. Checkout, setup Node 22, `npm ci`
2. Lint, typecheck, test, build
3. `docker build`
4. Start the container, wait, `curl` the `/health` endpoint (this is the deployment smoke test)
5. Validate the Kubernetes manifests (for example with kubeconform, or `kubectl apply --dry-run=client`)

No real cluster is needed. The container smoke test and manifest validation stand in for a deployment.

A5. Verify the baseline

- [ ] Push to `main`. The workflow must be **green**.
- [ ] If it is not green, fix it before making any failures.

A6. Create the failures For each of the 20 rows in `docs/product/PLAN.md`:

- [ ] Create a branch from `main`, make exactly one change that causes that failure, and confirm the workflow goes red for the intended reason.
- [ ] Save the change: `git diff main > failures/NN-slug/patch.diff`.
- [ ] Write `failures/NN-slug/meta.json`:

```json
{
  "id": "01",
  "slug": "typo-import",
  "class": "code",
  "expectedOutcome": "fix",
  "description": "Import path has a typo in src/index.ts",
  "expectedFiles": ["app/src/index.ts"]
}
```

- [ ] Ideas for the non-code rows: #18 pull a private image that returns "denied" for registry authentication; #19 a step that exits with "DEPLOY_TOKEN is not set"; #20 a step that fetches from an unreachable address with a short timeout to imitate a network flake.
- [ ] Delete the manual branch. Branches are always generated from the patches so they can be recreated.

A7. Scripts

- [ ] `verify-patches.sh`: apply every patch to a clean checkout of `main` and fail if one does not apply.
- [ ] `create-branches.sh`: for each failure folder, create `fail/NN-slug` from `main`, apply the patch, commit, push.
- [ ] `reset-branches.sh`: delete remote `fail/*` branches and recreate them (use after changing `main`).
- [ ] CI job in this repo that runs `verify-patches.sh` on every PR to `main`.
- [ ] Document usage in the testbed README.

Part B. CI and test foundations for the main repo

- [ ] `ci.yml`: install with pnpm cache, lint, typecheck, test, build. Add a Postgres service and run `prisma migrate deploy` against the empty database.
- [ ] Test utilities: fake GitHub webhook payloads and signature helper, a fake `GitHubClient`, a test database setup (Docker Postgres, reset between tests).
- [ ] Integration tests for the webhook endpoint with Dev 1.
- [ ] Coverage report in CI (no hard threshold yet).

Part C. Eval harness (`apps/eval` in the main repo)

- [ ] CLI: `pnpm eval run --ids 01,02` (default all).
- [ ] For each failure: trigger the workflow run on its `fail/NN-slug` branch (re-run through the GitHub API), wait for the platform to process it (poll the `failures` table), collect classification, outcome, and PR URL.
- [ ] Compare with `meta.json`. Write a row per failure to `eval-results/YYYY-MM-DD.csv` using the scoring fields in `docs/product/PLAN.md`.
- [ ] Print a summary: classification accuracy, outcome accuracy, validated-fix rate, secret leaks, median time.
- [ ] Keep result files in git so progress is visible over time.

Part D. Sandbox validator (weeks 4-5)

- [ ] Input: repo, commit, patch. Check out the commit in a temporary directory, apply the patch, run the same steps as the workflow (install, lint, typecheck, test, build, docker build).
- [ ] Run inside a container with limits: CPU, memory, time, and no access to secrets or the internal network.
- [ ] Output: pass or fail per step plus a short summary for the PR description.
- [ ] Tests with one known-good patch and one known-bad patch.

Definition of done

`pnpm eval run` produces a scored table for all 20 failures, the testbed can be rebuilt from patches with one script, and CI blocks broken PRs in the main repo.

---

8. Common problems

- **Webhooks do not arrive:** check the tunnel URL in the GitHub App settings and the "Recent deliveries" tab (you can redeliver from there).
- **Signature check fails:** the raw request body must be used, not the parsed JSON.
- **Workflow does not run on a failure branch:** confirm the `fail/**` trigger and that Actions are enabled for the repo.
- **Rebuilt branches differ from the app's data:** after resetting failure branches, old failure rows in the database refer to old commit ids. Clear them with `pnpm db:reset`.